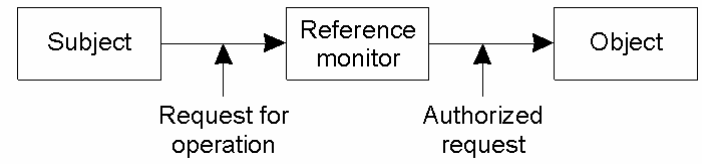
Securitatea sistemelor distribuite
Partea a 2-a

Controlul Accesului

Controlul acesului este bazat pe 2 premise:
- identificarea corecta a utilizatorului astfel incat nici un utilizator sa nu poata lua drepturile de acces ale altuia; identificarea este asigurata prin autentificare;
- protejatarea informatiei despre drepturile de acces contra modificarilor neautorizate.

Controlul Accesului – Elemente de baza

Intrebarea la care trebuie sa raspunda controlul accesului este urmatoarea: "Are subiectul **s** dreptul de access **a** la obiectul **o**?" Tuplul (s, o, a) constituie **autorizatia**. Controlul accesului este o functie f (s, o, a) care intoarce *true* sau *false*. Aceasta functie este implementtata de **Reference monitor**, o componenta a sistemului de securitate care primeste cererea de executie a operatiei **o** si decide daca operatia poate continua, caz in care paseaza obiectului cererea **autorizata**, sau daca cererea trebuie rejectata.



Clasificarea modelelor de acces

Desi problema Reference monitorului pare simpla, rezolvarea ei a evidentiat mai multe modele de securitate aplicabile. Reamintim ca un model de securitate este o reprezentare mai precisa si mai detaliata a unei politici de securitate. Modelul poate fi folosit ca referinta pentru construirea securitatii si pentru evaluarea ei. Traditional, modelele de acces sunt clasificate dupa doua criterii.
Primul criteriu este posibilitatea de transfer al drepturilor, care `imparte modelele in discretionare si mandatorii. Modelele discretionare - Discretionary Access Control (DAC) permit oricarui utilizator sa transfere altora drepturile pe care le detine, la discretia lor. De exemplu, proprietarul unui fisier poate da altor utilizatori dreptul de citire a fisierului respectiv. Modelele mandatorii - Mandatory Access Control (MAC) nu permit utilizatorilor sa transfere drepturile pe care le detin.
Al doilea criteriu de clasificare este obiectul controlat. Acesta poate fi fluxul de informatii, asa cum acesta este definit si folosit in modelele multinivel sau accesul la informatii, asa cum acesta apare in modelele matricei de acces sau in RBAC – Role Based Access Control.

Model de confidentialitate multinivel

In modelele de confidentialitate multinivel, fiecare obiect are un nivel de senzitivitate sau **rang**, cum ar fi:

neclasificat
restrictionat
confidential

secret
top secret.

De asemenea, fiecare obiect este asociat cu unul sau mai multe proiecte (compartimente), accesul la obiect fiind permis celor care "trebuie sa stie" pentru ca lucreaza in proiectul respectiv.

Clasa de acces a unui obiect este combinatia <rang, compartimente>

Fiecare subiect are o autorizare (clearance) exprimata ca o clasa de acces <rang, compartimente> ce specifica:

la ce rang de senzitivitate are acces respectivul subiect si
in ce proiecte lucreaza.

Relatia de **dominanta** intre clase de acces este definita astfel :

Ci domina Ck (sau Ck este dominat de Ci)

$Ci \geq Ck \Leftrightarrow$

rang (Ci) $\geq$ rang (Ck) si
compartimente (Ci) □ compartimente (Ck)

Ci domina strict Ck, $Ci > Ck \Leftrightarrow$

rang (Ci) > rang (Ck) si
compartimente (Ci) □ compartimente (Ck)

Ci si Ck sunt incomparabile Ci <> Ck daca

nici Ci $\geq$ Ck nici Ck $\geq$ Ci

Exemple

Fie clasele de acces definite astfel:

C1 = (TopSecret, {Nuclear, Armata})
C2 = (TopSecret, {Nuclear})
C3 = (Confidential, {Armata})

Avem urmatoarele relatii intre aceste clase:

C1 $\geq$ C2	deoarece au acelasi rang si {Nuclear, Armata} include strict {Nuclear}
C1 > C3	deoarece TopSecret > Confidential si {Nuclear, Armata} include strict {Armata}
C2 < > C3	

Modelul Bell-La Padulla (BLP)

BLP este un model de confidentialitate multi-nivel. Modelul pastreaza secretul, prevenind divulgarea neautorizata a informatiei. De aceea, moelul are aplicabilitate, in primul rand, in sistemele de securitate militare.

Modelul este definit prin doua reguli pe care le prezentam succint in continuare.

Securitatea simpla (*no-read-up*): Subiectele au acces doar la informatia pentru care au clasa de acces necesara.

Formal: Un subiect s are acces *read* la un obiect o doar cand clasa sa de acces domina clasa obiectului

$C(s) \geq C(o)$

Proprietatea Star (*) (*no-write-down*) Previne fluxul de informatie inspre obiecte cu clase de acces inferioare sau incomparabile.
Formal: Un subiect s care are acces *read* la un obiect p, poate avea acces *write* la un obiect o doar cand clasa de acces a lui o domina clasa lui p
$C(o) \geq C(p)$

Exemplu
Fie urmatoarele specificatiii de subiecti si obiecte.

Rang	**Subiect**	**Obiect**
Top secret	Ion, Rodica	Fisiere de personal
Secret	Sanda, Vasile	Fisiere e-mail
Confidential	Costi, Ioana	Fisiere log
Neclasificat	Mara, Mihai	Fisiere numere telefon

Senzitivitatea scade de sus in jos.
In acest exemplu, securitate simpla stabileste urmatoarele posibilitati de acces:
Ioana nu are acces la Fisiere de personal
Rodica are acces la toate fisierele

Proprietatea-* determina ca
Rodica poate citi Fisiere de personal dar nu le poate scrie in Fisiere log pentru accesul Ioanei.

Modelul de integritate Biba

Folosind reguli similare, au fost definite modele de integritate, dintre care modelul Biba este unul reprezentativ. El defineste nivele de integritate I analoage nivelelor de senzitivitate din modelul BLP si urmatoarele reguli de integritate.

Integritatea simpla
Subiectul s poate modifica (acces *write*) un obiect o care are integritatea mai mica
$I(s) \geq I(o)$
Justificare: un subiect s cu integritate mai mica ar scadea integritatea obiectului o modificat de el

Proprietatea-*
Daca subiectul s are acces *read* la obiectul p cu integritatea I(p), atunci el poate avea acces *write* la obiectul o doar daca
$I(p) \geq I(o)$
Justificare: un obiect p cu integritatea mai mica ar scadea integritatea obiectului o

Modelul are aplicabilitate in securizarea sistemelor comerciale, unde informatiile sunt accesibile publicului dar nu trebuie modificate de persoane ne-autorizate. De aici rezulta utilitatea unui model care asigura integritatea obiectelor, fiind mai putin importanta confientialitatea loc.

Caracteristici comune

Modelele Bell La-Padula si Biba au trasaturi comune:

nu specifica modul de definire sau de modificare a claselor de acces si de autorizare

nu trateaza delegarea sau transferul drepturilor de acces.

Aceste topici sunt tratate de alte modele cum ar fi Graham-Denning a carui idee generala este definirea unei matrice de acces care sa specifice drepturile de acces pentru fiecare combinatie de subiecte si obiecte.

Cu Graham-Denning trecem la o alta categorie de modele care permit determinarea limitelor teoretice ale sistemelor de securitate sau, cu alte cuvinte, demonstrarea fezabilitatii unor solutii de securitate care au anumite proprietati dorite.

Modelul Matricei de Acces

Modelul este definit in termeni de stari si tranzitii:

- o stare este reprezentata de o matrice
- tranzitiile sunt descrise prin actiuni asupra elementelor matricei.

Modelul considera

o multime de subiecte S = {s1,...,sn}

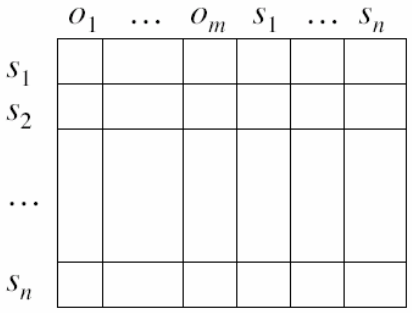
o multime de obiecte O = {o1,...,om}

o multime de drepturi R = {r1,...,rk}

o matrice de acces ale carei intrari A [s,o] $\subseteq$ R precizeaza ca subiectul s are drepturile {r1,...,rk} asupre obiectului o.

In modelul **Graham-Denning**, drepturile sunt definite ca

- actiuni A [s,o] pe care subiectul s le poate executa asupra obiectului o
- actiuni A[si,sj] pe care subiectul si le poate executa asupra subiectului sj



Exemplu

Drepturile proceselor P1 si P2

asupra fisierelor f1 si f2

si asupra proceselor P1 si P2

sunt prezentate in tabelul urmator.

	f1	f2	P1	P2

P1	read write own	read	read write execute own	write
P2	append	read own	read	read write execute own

In modelul Graham - Denning, fiecare obiect are un subiect **propietar** (owner) cu drepturi speciale si fiecare subiect are un alt subiect cu drepturi speciale (**controlor**). Modelul defineste o serie de actiuni primitive pe care un subiect x le poate executa asupra obiectului o sau subiectului s. Ele sunt prezentate in tabelul urmator in care apar preconditiile impuse aplicarii actiunilor si efectele executiei lor.

Actiune	**Preconditie**	**Efect**
create object o	-	Adauga coloana o la A; adauga *Owner* la A[x,o]
delete object o	*Owner* in A[x,o]	Sterge coloana o
create subject s	-	Adauga linia s si col s la A; adauga *Control* la A[x,s]
delete subject s	*Control* in A[x,s]	Sterge linia s si col s
read access rights of s on o	*Control* in A[x,s] sau Owner in A[x,o]	Citeste A[s,o]
grant access right r to s on o	*Owner* in A[x,o]	Adauga r la A[s,o]
delete access right r of s on o	*Control* in A[x,s] sau *Owner* in A[x,o]	Sterge r din A[s,o]
transfer access right r or r to s on o*	r* in A[x,o]	Adauga r sau r* la A[s,o]

Actiuni primitive executate de subiectul x: r* inseamna ca dreptul **r** transmis de **x** lui **s** este *transferabil*, adica s poate transfera r sau r* altor subiecte.
Acest set de reguli modeleaza mecanismele de control al accesului intr-un sistem de protectie si poate sta la baza realizarii unui Reference monitor.

Alte modele – HRU

Modelul Harrison-Ruzzo-Ullman (HRU) este o varianta extinsa a modelului Graham-Denning si ofera raspunsul la intrebari privitoare la felul de protectie pe care un sistem il poate oferi. HRU este bazat pe aceleasi elemente ca Graham-Denning:

S - set de subiecte
O - set de obiecte
R - set de drepturi de acces

O - matrice de acces $M = (M_{so})_{s \in S,\ o \in O}$

Intrarea M_{so} este un subset din R specificand drepturile subiectului s asupra obiectului o.
Modelul defineste insa o multime restransa de operatii primitive:

enter right r into M_{so}
delete right r from M_{so}
create subject s
delete subject s
create object o
delete object o

Comenzi

Aceste operatii primitive pot fi combinate in comenzi, care au formatul:

command $c(x_1,.....,x_k)$
if r_1 in $M_{s1,o1}$ **and**
if r_2 in $M_{s2,o2}$ **and**
…
if r_m in $M_{sm,om}$
then $op_1,.....,op_n$
end

unde $s_1,.....,s_m$ si $o_1,.....,o_m$ sunt subiecte si obiecte care apar in lista de parametri $x_1,.....,x_k$

Executia unei comenzi se face astfel: daca toate conditiile sunt indeplinite atunci se executa lista de operatii.
In particular, pot fi definite comenzi mono-operatii (lista de comenzi contine o singura operatie).

Exemple

Crearea unui fisier:

command CREATE_FILE(s,o)
create object o
enter own into *A[*s,o*]*
end CREATE_FILE

Transferul unui drept; de ex "read":

command CONFER_READ(s1,o,s2)
if own ∈ *A[*s1 ,o*]*
then enter read into *A[*s2 ,o*]*
end CONFER_READ

Revocarea unui drept; de ex "write":

command REVOKE_WRITE(s1,o,s2)
if (own ∈ *A[*s1 ,o*]*) and (write ∈ *A[*s2 ,o*]*)
then delete write from *A[*s2 ,o*]*
end REVOKE_WRITE

Ce rezolva HRU - Sisteme de Protectie

Un sistem de protectie este definit ca un ansamblu de seturi finite de subiecte, obiecte, drepturi si comenzi.
Un sistem de protectie este unul stari-tranzitii, in care fiecare stare este descrisa de o valoare a matricei de acces, iar tranzitia de la o stare la alta este determinata de executia unei comenzi. Una din problemele fundamentale in sistemele de protectie este scurgerea neautorizata a drepturilor de acces. Definitiile urmatoare se refera la notiunile de scurgere a unui drept si de stare de protectie sigura.

Definitie. O stare M lasa sa se *scurga* dreptul *r* daca exista o comanda *c* care adauga dreptul *r* intr-o intrare din *M* care anterior nu continea *r*. Mai precis,
exista *s* si *o* astfel ca $r \in M_{so}$ si
dupa executia lui *c*, $r \in M'_{so}$
Definitie. O stare *M* intr-un sistem de protectie este **sigura** relativa la un drept *r* daca nici o secventa de comenzi nu poate transforma *M* intr-o stare in care se scurge *r*.
Nota: Faptul ca un drept se poate "scurge" nu este neaparat rau; multe sisteme permit subiectilor sa delege drepturi de acces altor subiecti

Exemplu de sistem "nesigur"

Exemplu
Bob a dezvoltat o aplicatie si vrea ca ea sa fie executata de alt utilizator dar nu modificata de acesta
Sistemul de protectie include urmatoarele comenzi:

command grant_execute (*s*,*p*,*f*)
if own in $M_{s,f}$
then enter execute into $M_{p,f}$
end

command modify_own_right (*s*,*f*)
if execute in $M_{s,f}$
then enter write into $M_{s,f}$
end

Sistemul anterior este nesigur deoarece permite executia urmatoarei secvente de comenzi:
- Bob: grant_execute (Bob, Tom, P1)
- Tom: modify_own_right (Tom, P1)

care face ca in M, intrarea $M_{\text{Tom,P1}}$ sa contina dreptul de acces w care da dreptul lui Tom sa modifice aplicatia dezvoltata de Bob.

Siguranta in modelul HRU

Una din intrebarile fundamentale se refera la verificarea sigurantei unei stari intr-un sistem de protectie. Raspunsul este dat de urmatoarea
Teorema. Data fiind *M* si dreptul *r*, verificarea sigurantei lui *M* relativa la *r* este o problema nedecidabila in cazul general.

Problema verificarii sigurantei poate fi reformulata si astfel:

Este decidabil daca un *subiect* ar putea obtine vreodata un anumit *drept* relativ la un *obiect* ?

Faptul ca verificarea sigurantei este in general nedecidabila, nu inseamna ca un este posibila luarea unor decizii pentru sisteme de protectie particulare. Rezultatele bazate pe HRU au aratat ca problema sigurantei este decidabila pentru sisteme de protectie mono-operatie. Ea nu este intotdeauna decidabila pentru alte tipuri de sisteme de protectie. In cazul sistemului de operare UNIX, protectia cere mai mult de o operatie per comanda, deci nu exista o procedura generala pentru a determina daca un anumit drept de acces poate fi dat unui subiect.

Modelul Take-Grant

Acest model ofera suportul unei capacitati sporite de decizie in problema protectiei. Modelul are doar patru operatii primitive, care sunt reprezentate prin grafuri. Nodurile reprezinta subiecti si obiecte, iar drepturile sunt reprezentate prin arce etichetate de la subiect la obiect.

Urmatoarele operatii sunt executate de s.

create(o, r):



revoke(o,r):



grant(o,p,r):



Prin aceasta operatie, subiectul s “deleaga” lui o dreptul de acces r asupra lui p. Operatia are urmatoarele preconditii:

s are drept de delegare (Grant) a unor drepturi catre o

s are dreptul r asupra lui p

take(o,p,r):



Prin aceasta operatie, subiectul s preia de la o dreptul de acces r asupra lui p

Preconditii :

s are drept de preluare (Take) de drepturi de la o

o are dreptul r asupra lui p

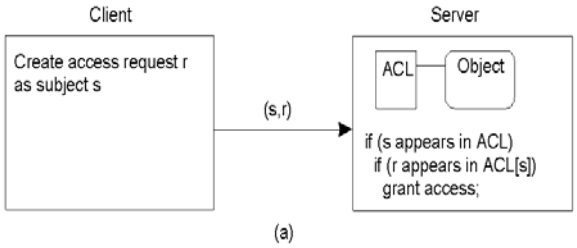
Cu acest model se poate decide daca:

- un subiect poate partaja un obiect cu un alt subiect si daca
- un subiect poate "fura" accesul la un obiect de la un alt subiect.

Modelul este util pentru ca identifica conditiile in care un subiect poate obtine anumite drepturi de acces la un obiect.

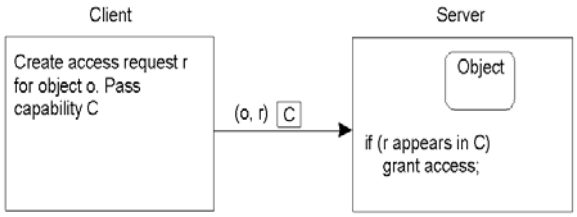
Implementare Matrice Control Acces

Traditional, matricile de acces sunt implementate prin liste de control al accesului ACL - Access Control List. Fiecare obiect pastreaza o coloana a matricei de acces, MCA[*,O]



(a)

Ca alternativa, se poate folosi modelul de Capabilitati, in care fiecare subiect are capabilitatile specificate de o linie a matricei de acces, MCA[S,*]



(b)

Probleme cu modelele "clasice"

Modelele clasice (matrice de acces, respectiv ACL, C-list) au deficientele mentionate in continuare.

Ele nu pot reprezenta modele de acces mai complexe, de exemplu modele bazate pe reguli cum ar fi competenta subiectilor. De asemenea, nu pot surprinde aspecte cum sunt cele mai reduse privilegii (necesare pentru realizarea procesarilor care intra in sarcina subiectului) sau evitarea conflictul de interese (rezultat din detinerea de catre un acelasi subiect a unor drepturi de acces incompatibile (de exemplu, intr-o organizatie, un subiect sa poata cere efetuarea unei cheltuieli si in acelasi timp sa o poata aproba).

Modelele nu suporta cu usurinta schimbari dinamice: astfel, modificarea drepturilor unui subiect necesita inspectarea ACL a fiecarui obiect; determinarea subiectilor care au acces la un obiect necesita inspectarea tuturor listelor de Capabilitati.

Modelele nu pot gestiona drepturi de acces determinate de continutul, atributele obiectelor sau de context. In fine, modelele nu inregistreaza "consumul" permisiunilor, un subiect putand utiliza permisiunile ori de cate ori doreste.

RBAC - Role Based Access Control

Un model care elimina unele din neajunsurile mentionate este RBAC. Rolul este un mecanism de grupare a subiectilor dupa anumite atribute: ocupatie, responsabilitati, functii. In RBAC, rolurile (si nu subiectii) au asociate permisiuni ceea ce are urmatoarele avantaje:

- permite predefinirea de roluri, asignarea lor putandu-se face ulterior in functie de necesitati;
- deoarece relatia rol – permisiuni se schimba mai rar decat subiet – rol, rezulta o administrare mai usoara a drepturilor de acces prin schimbarea rolului asociat unui utilizator.

Fiecare subiect are un set de roluri asignate. Relatia este caracterizata de urmatoarele:

- asignarea / revocarea de roluri poate fi administrata usor;

- un utilizator poate trece de la un rol la altul intr-o sesiune;
- un utilizator poate avea mai multe roluri simultan.

Rolurile sunt supuse unor reguli suplimentare cum ar fi separarea statica sau dinamica a obligatiilor legale (constrangeri). Aceasta face ca, de exemplu, o persoana sa nu poata initia o plata si autoriza aceeasi plata. Constrangerile pot lua doua forme:

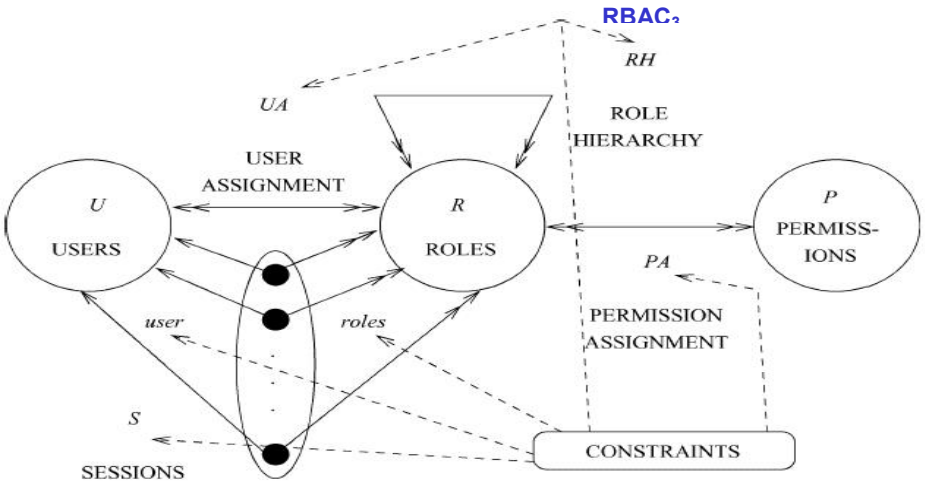
- Static Separation of Duty Relations – de ex. rolurile A si B sa nu fie ambele asignate unui utilizator
- Dynamic Separation of Duty Relations - de ex. rolurile A si B sa nu fie active in acelasi timp pentru un utilizator

Rolurile pot fi ierarhizate. De exemplu, un instructor poate avea drepturile unui student si in plus alte drepturi. Ierarhia este o relatie de ordine partiala intre roluri. Ierarhizarea simplifica gestiunea drepturilor (de ex. prin mostenire).
Sunt doua categorii de modele ierarhice:

- limitate (ex. ierarhii arborescente) sau
- generale (ex. mostenire multipla).

RBAC – Familie de modele

RBAC este o familie de modele. Modelul de baza – Core RBAC – este definit de RBAC0 si raspunde cerintelor minime ale unui sistem RBAC.



RBAC1 - Hierarchical RBAC defineste ierarhiile de roluri (relatii de ordine intre roluri) in care un rol poate mosteni permisiuni din alte roluri.
RBAC2 - Static & Dynamic Separation of Duty Relations defineste constrangeri asupra configuratiilor de roluri.
RBAC3 – include RBAC1 si RBAC2 si, prin tranzitivitate, RBAC0.

Prezentam in continuare modelul de baza, facand referiri apoi la ierarhizarea rolurilor si la specificarea constrangerilor. Scopul principal al prezentarii este intelegerea conceptelor incorporate in modelele RBAC. In plus, se urmareste intelegerea modului de specificare a unui model de control al accesului, astfel incat aceasta sa fie o referinta pentru compararea cu alte modele, precum si pentru dezvoltarea si evaluarea unor produse care respecta standardul RBAC.

Model de referinta pentru Core RBAC

Modelul de referinta pentru RBAC0 are urmatoarele componente:

- o multime de seturi de elemente de baza
- un set de relatii RBAC pe aceste seturi (asocierile valide) si
- un set functii de mapare care dau instantele de membri dintr-un set care corespund unui membru din alt set

Seturile de elemente de baza, *USERS, ROLES, OPS* (operatii), *OBS* (obiecte), *SESSIONS* (set de sesiuni) au urmatoarele semnificatii

User – persoana, masina, proces, agent software, etc.

Role – functie in contextul unei organizatii, asociata cu autoritatea si responsabilitatile sale

Permission – aprobare a unui anumit mod de acces la unul sau mai multe biecte ale sistemului (similar cu autorizare, drept de acces, privilegiu)

Session – instanta particulara a unei conexiuni a utilizatorului cu sistemul, in care utilizatorul poate activa un subset din rolurile ce ii sunt asignate; sesiunea este sub controlul total al utilizatorului care creaza si termina sesiunea si stabileste ce roluri activeaza in acea sesiune.

Relatiile si functiile de mapare sunt urmatoarele:

User Assignment – UA $\subseteq$ *USERS* $\times$ *ROLES*; relatie many-to-many intre utilizatori si rolurile asignate; un utilizator poate avea asignate mai multe roluri; un rol poate fi asignat mai multor utilizatori;

assigned_users(r:ROLES) = $\{u \in USERS \mid (u, r) \in UA\}$ mapeaza un rol pe multimea utilizatorilor care au acel rol;

Permissions – PRMS = $2^{(OPS \times OBS)}$; reprezinta multimea permisiunilor; o permisiune desemneaza o multime de operatii care se aplica unei multimi de obiecte;

permisiuni $\rightarrow$ operatii - *Op(p: PRMS)* $\rightarrow$ {op$\subseteq$OPS} mapeaza o permisiune pe multimea operatiilor

permisuni $\rightarrow$ obiect - *Ob(p: PRMS)* $\rightarrow$ {ob$\subseteq$OBS} mapeaza o permisiune pe multimea obiectelor corespunzatoare

Permission Assignment - PA $\subseteq$ *PRMS* $\times$ *ROLES*; relatie many-to-many intre permisiuni si roluri; o permisiune poate fi asignata mai multor roluri; un rol poate avea mai multe permisiuni asignate;

assigned_permissions(r : ROLES) $\rightarrow 2^{PRMS}$ mapeaza un rol pe multimea permisiunilor asignate

user_sessions (u: USERS) $\rightarrow 2^{SESSIONS}$ mapeaza un utilizator pe multimea sesiunilor sale

session_user (*s : SESSIONS)* $\rightarrow$ USERS mapeaza o sesiune s_i pe singurul utillizator *session_user* (s_i)

session_roles (s: SESSIONS) $\rightarrow 2^{ROLES}$ mapeaza o sesiune pe multimea rolurilor data de *session_roles* (s_i) $\subseteq$ $\{r \in ROLES \mid (session_user\ (s_i), r) \in UA\}$

avail_session_perms(s:SESSIONS) $\rightarrow 2^{PRMS}$ mapeaza o sesiune pe multimea permisiunilor disponibile utilizatorului in acea sesiune =

$$\bigcup_{r \in session_roles(s)} assigned_permissions(r)$$

Politici de activare - modelul defineste doua politici de activare a rolurilor si anume :
Single-role activation (SRA) = un singur rol poate fi activat intr-o sesiune ;
Multi-role activation (MRA) = mai multe roluri pot fi activate intr-o sesiune.
In al doilea caz, se pot folosi constrangeri "separation of duty" pentru a restrictiona activarea concurenta a anumitor roluri.

Ierarhia de roluri
Un rol poate include alte roluri. de exemplu, un instructor are permisiunile studentilor si, in plus, alte permisiuni. Spunem ca rolul ri include rolul rk si notam aceasta ri > rk daca si numai daca RolePermissions (ri) $\supseteq$ RolePermissions (rk) si
AssignedUsers (rk) $\supseteq$ AssignedUsers (ri).

Separarea sarcinilor (excludere mutuala a permisiunilor)
RBAC admite existenta rolurilor incompatibile, care nu pot fi asumate simultan de un utilizator. Prin definitie, daca ri, rk sunt incompatibile atunci doar unul poate fi asignat unui acelasi utilizator. Deci, ri $\in$ AssignedRoles(u) => rk $\notin$ AssignedRoles(u).

Functii administrative si de sistem

In afara componentelor descrise anterior, functionarea unui sistem presupune existenta unor functii administrative si de sistem supuse unui model administrativ RBAC. Aici facem doar o descriere sumara a acestor functii.

Functii Administrative
Creare si Intretinere seturi de elemente
Creare si Intretinere Relatii

Functii Suport Sistem - Management sesiune
CreateSession - Creaza User Session si da utilizatorului un set de roluri active implicite
AddActiveRole – Adauga un rol ca rol activ in sesiunea curenta
DropActiveRole – Elimina un rol activ din sesiunea curenta
Decizii de control al accesului
CheckAccess – Determina daca subiectul are permisiunea sa execute o anumita operatie asupra obiectului.

Functii Administrative de informare (Review) In enumerarea urmatoare se specifica functiile obligatorii M – Mandatory si functiilr optionale O – Optional.
AssignedUsers (M) - Intoarce setul de utilizatori asignat unui rol dat
AssignedRoles (M) - Intoarce setul de roluri asignate unui utilizator dat
RolePermissions (O) - Intoarce setul de permisiuni garantate unui rol dat
UserPermissions (O) - Intoarce setul de permisiuni pe care un utilizator le are prin rolurile sale
SessionRoles(O) - Intoarce setul de roluri active asociate cu o sesiune

SessionPermissions (O) - Intoarce setul de permisiuni disponibile in sesiune (reuniunea tuturor permisiunilor asignate rolurilor active din sesiune)

Caracteristici RBAC

RBAC are urmatoarele calitati:
- utilizatorul poate trece usor de la un rol la altul in cursul unei sesiuni, fara a schimba structura accesului
- ca rezultat, RBAC mai scalabil ca matricile de acces si variantele lor (ACL si C-List)
- permite tratarea conflictelor de interese (rolurile au reguli stricte)
- reduce overhead-ul la administrarea securitatii la nivel de utilizator, obiect, permisiune.

Relatia cu MAC si DAC
RBAC este un model neutru fata de politicile de control al accesului si poate coexista cu MAC sau DAC. In cazul utilizarii in tandem a RBAC si a unei politici MAC sau DAC, efectele regulilor de control al accesului se cumuleaza: accesul trebuie sa fie permis de RBAC si de MAC / DAC.

Utilizare
RBAC este utilizat in diverse produse software comerciale sau deschise cum ar fi Microsoft Active Directory, SELinux, FreeBSD, Solaris, Oracle DBMS, PostgreSQL 8.1, SAP R/3.